

You're at a **great inflection point** now.

You've already crossed what most "Redis clone" projects never do:

- TTL engine (heap + background thread)
- Lazy + proactive expiry
- Observability (stats, hotkeys, slowlog, memory)
- Correct RESP, data structures, concurrency

From here, the next features should **increase real-world usefulness**, not just add commands.

I'll give you a **ranked roadmap** with *why* each matters and *what to implement*.

Tier 1 — Must-have (Production credibility)

1 Key eviction policies (LRU / LFU)

Right now your DB grows forever.

Why this matters

- Redis without eviction is unusable in prod
- Perfect tie-in with your `key_access_count`

What to implement

- Configurable max memory (e.g. `MAXMEMORY 64mb`)
- Eviction strategies:
 - o `allkeys-lru`
 - o `allkeys-lfu` (you're 80% done already)
- On `SET` / `LPUSH` / `SADD`, check memory → evict

You already have:

- `key_access_count` 
- memory estimation 

This is a *low effort / high impact* feature.

2 Persistence (RDB snapshot OR AOF)

Your data disappears on restart.

Start simple: RDB snapshot

What to implement

- Background thread every N seconds:
 - Serialize db to file (bincode or custom format)
- On startup:
 - Load snapshot if exists

Bonus

- Add SAVE and BGSAVE commands

This instantly moves your project from *toy* → *real system*.

3 Config system

Right now everything is hardcoded.

Add

- CONFIG GET
- CONFIG SET

For:

- maxmemory
- slowlog threshold
- expiry tick interval

This shows **systems thinking**.

Tier 2 — High-value differentiators

4 Pub/Sub

This is surprisingly easy and very impressive.

Commands

- SUBSCRIBE channel
- PUBLISH channel message

Implementation idea

- `HashMap<String, Vec<TcpStream>>`
- Fan-out messages to subscribers

This shows **network + concurrency mastery**.

5 SCAN (cursor-based iteration)

Avoids blocking the server.

Commands

- SCAN cursor
- Optional MATCH, COUNT

Why this matters

- Shows non-blocking design
 - Redis interview classic
-

6 Lua-style scripting (lite)

Not full Lua — keep it sane.

Idea

- EVAL with a tiny expression engine
- Or Rust-based scripting (closure-like)

This is advanced and optional, but very impressive.

Tier 3 — Advanced / “This person is scary good”

7 Replication (primary → replica)

Even basic replication is huge.

Start simple

- Primary streams commands
- Replica replays them

This is *hard*, but unbeatable for resumes.

8 Transactions

- MULTI
- EXEC
- DISCARD

Even without rollback — queue + execute is enough.

9 ACL / AUTH

- AUTH password
- Command restrictions per client

Security awareness = maturity.

What I recommend YOU do next (in order)

Given your current state, do **exactly this**:

✅ Step 1 (next 1–2 days)

Eviction policy (LFU first)

You already have 70% of it done.

✅ Step 2

RDB snapshot persistence

✅ Step 3

CONFIG + tunables

After this, your project is no longer “Redis clone” —
it’s a **key-value database with observability-first design**.



Resume-ready one-liner (already true)

Built an in-memory key-value database in Rust with TTL, background expiry engine, observability (hotkeys, slowlog, memory stats), and custom RESP protocol support.

After eviction + persistence, this becomes **elite**.

If you want, next I can:

- design **LFU eviction code with zero new structs**
- help you choose **RDB vs AOF**
- prepare **final architecture diagram**
- help turn this into a **GitHub README that pops**

Just tell me which feature you want to attack next.